

2025秋计算机体系结构 作业2

概述

现代多核计算机系统大多在硬件层面支持共享内存机制。在共享内存系统中，每个处理器内核都能对统一的共享地址空间进行读写操作。缓存一致性协议作为管理各级缓存数据读写权限的重要机制，对保障多核系统中共享缓存的正确运行起着关键作用。

设计缓存一致性协议面临严峻挑战，这主要源于协议中存在的多种竞态条件和边界情况。虽然目前已开发并验证了若干复杂协议，但协议验证作为设计流程中不可或缺却又极其困难的环节，仍是学术界持续探索的前沿领域。本次作业要求我们为多核系统设计并验证一套缓存一致性协议。

我们将采用形式化验证工具 Murphi 进行协议验证。该工具通过模型检测技术，在给定系统有限状态模型和期望属性的前提下，自动检验这些属性是否在模型所有可达状态中均成立。具体而言，我们需要用 Murphi 描述语言构建与一致性协议对应的有限状态机，并定义一组期望属性（即不变式）。Murphi 验证器会系统地枚举所有可达状态空间，检查指定的不变式是否未被违反。

文件说明

- 用户手册：
Murphi 的使用说明文档，位于 `Murphi3.1/doc/User.Manual`，可用于查阅 Murphi 语言的语法与建模示例。
- 示例程序：
位于 `Murphi3.1/ex/toy` 目录中，包含若干 Murphi 示例模型，便于熟悉 Murphi 描述语言与验证流程。
建议首先阅读 `pingpong.m` 示例，了解 Murphi 模型的基本结构和语法用法。
- 代码框架：
位于 `Murphi3.1/lab2` 目录中，提供实验所需的协议模型代码框架，请在此基础上补充完整实现。

实验内容

在本次实验中，你将使用形式化验证工具 **Murphi** 实现并验证一个基于侦听的 **MSI** 缓存一致性协议。该协议应具备以下特征：

1. 基于广播的共享总线结构

- 协议通过一条共享总线（Bus）实现缓存与内存之间的通信。
- 任何读/写请求都会在总线上广播，所有缓存同时进行监听并响应。

2. 多缓存系统结构

- 系统包含 3 个缓存（Cache），每个缓存属于不同的处理器。
- 每个缓存只包含 1 个大小为 1 个内存单元的缓存行（为简化验证），但能够模拟读写行为。
- 各缓存通过一致性协议保持对同一内存块数据的一致视图。

3. 共享主存模块

- 系统中仅包含唯一的主存模块，内含 1 个内存单元（为简化验证）。
- 当总线上没有缓存响应数据时，由主存向请求方提供数据。

4. 三种缓存状态

- **M (Modified)**：缓存拥有数据的唯一有效副本，数据可能与内存不一致。
- **S (Shared)**：缓存拥有数据的共享副本，数据与内存一致。
- **I (Invalid)**：缓存行无效，需从其他缓存或内存获取最新数据。

5. 处理器行为建模

- 处理器可发出读（PrRd）和写（PrWr）请求。
- 若请求命中缓存，直接在缓存中读写；若缺失，则在总线上广播请求。

6. 两类总线事务

- **BusRd**：由读缺失触发，请求共享数据副本。
- **BusRdX**：由写缺失或写升级触发，请求独占数据副本。

7. 缓存侦听机制

- 所有缓存控制器通过侦听总线信号来感知其他缓存的请求，并对总线请求做出响应。
- 假设内存控制器对总线事务的响应晚于各缓存的 snoop，即优先由缓存响应总线请求，若无缓存响应则由内存处理。

一、在进行协议编写之前，请根据你对 MSI 协议的理解，填写下面的 Cache 状态变化表格。

当前Cache状态	CPU操作	触发的总线事务	转移后的Cache状态
M	PrRd		
M	PrWr		
S	PrRd		
S	PrWr		
I	PrRd		
I	PrWr		

当前Cache状态	侦听到的总线事务	Cache执行的操作	转移后的Cache状态
M	BusRd		
M	BusRdX		
S	BusRd		
S	BusRdX		
I	BusRd		
I	BusRdX		

二、请在提供的 `msi.m` 代码框架基础上，依据 Cache 状态变换表，补充并完善 MSI 协议的关键逻辑实现，并使用 Murphi 对协议模型进行验证，以确保协议满足一致性不变式。

三、基于侦听 MSI 缓存一致性协议存在哪些问题？可能的改进方向是什么？

选作内容（加分）

使用形式化验证工具 Murphi 实现并验证一个基于侦听的 MESI 缓存一致性协议。

一、在进行协议编写之前，请根据你对 MESI 协议的理解，填写下面的 Cache 状态变化表格。

当前Cache状态	CPU操作	触发的总线事务	转移后的Cache状态
M	PrRd		
M	PrWr		
E	PrRd		
E	PrWr		
S	PrRd		
S	PrWr		
I	PrRd		
I	PrWr		

当前Cache状态	侦听到的总线事务	Cache执行的操作	转移后的Cache状态
M	BusRd		
M	BusRdX		
E	BusRd		
E	BusRdX		
S	BusRd		
S	BusRdX		
S	BusUpgr		
I	BusRd		
I	BusRdX		
I	BusUpgr		

二、请在提供的 `mesim` 代码框架基础上，依据 Cache 状态变换表，补充并完善 MESI 协议的关键逻辑实现，并使用 Murphi 对协议模型进行验证，以确保协议满足一致性不变式。

三、基于侦听 MESI 缓存一致性协议存在哪些问题？可能的改进方向是什么？

实验步骤

本次实验需要在 linux 环境下进行。

Step 0. 构建 Murphi

1. 首先安装所需依赖：

```
sudo apt update  
sudo apt install build-essential flex bison g++-multilib
```

2. 解压并编译 Murphi 源码：

```
tar -xvzf Murphi3.1.tar.gz  
cd Murphi3.1/src  
make mu
```

编译成功后，当前目录下会生成 `mu` 可执行文件，同时系统会在各目录下创建指向该文件的链接。

3. 验证 Murphi 正确性：

- 1) 进入 `ex/sci` 目录；
- 2) 使用命令 `./mu sci.m` 编译 SCI 协议；
- 3) 使用命令 `make sci` 构建 SCI 协议的验证器；
- 4) 使用命令 `./sci` 运行验证器；
- 5) 结果应为：“No error found”，共探索 18193 个 states 和 60455 条 rules。

Step 1. 补充并完善协议的关键逻辑实现

代码框架位于 `Murphi3.1/lab2` 目录中。

Step 2. 编译 Murphi 源文件

使用 `mu` 将 Murphi 源文件转换为 C 代码：

```
cd Murphi3.1/lab2
./mu msi.m
```

此时应生成一个名为 `msi.c` 的文件。

Step 3. 编译 Murphi 生成的C文件

使用 `make` 编译生成可执行文件：

```
make msi
```

执行成功后，目录中会出现可执行文件 `msi` 。

Step 4. 运行

运行模型并查看验证过程：

```
./msi
```

在实验过程中，可能需要使用 `./msi -tv` 来进行debug，详细说明见附录。

输出

```
=====
=====
Status:
  No error found.

State Space Explored:
  22423 states, 45713 rules fired in 0.10s.
```

作业提交要求

一、提交内容

最终提交的文件应包括以下两部分：

1. 实验报告（PDF 格式）

- 命名格式：`[姓名]-[学号]-lab2.pdf`，例如：`张三-23000001-lab2.pdf`
- 报告内容应包括：
 1. Cache 状态变换表格；
 2. 协议关键实现代码及说明；
 3. 执行 `./msi` 命令后的输出内容，显示未发现错误，同时包含已探索的状态数量以及运行时间；
 4. 思考题。

2. Murphi 源代码文件

- 文件名：`msi.m`，`mesi.m(选做)`
- 代码需能通过 Murphi 编译和运行，且逻辑与报告内容一致。

请将上述文件打包成一个 ZIP 压缩包，命名格式为：`[姓名]-[学号]-lab2.zip`，例如：`张三-23000001-lab2.zip`

二、提交方式

请将作业提交至超算习堂。

三、提交截止时间

12月7日 23:59

附录

1. Murphi 介绍

(1) Murphi 简介

Murphi 是一款显式状态协议验证器，由 Murphi 编译器和 Murphi 描述语言组成。

Murphi 编译器可根据 Murphi 描述生成专用验证器。该专用验证器能用于检查系统属性，包括错误断言、不变式和死锁情况。

Murphi 描述语言是一种用于有限状态异步并发系统的高级描述语言。其“高级”体现在，它包含了 Pascal 或 C 等常见高级编程语言的诸多特性。例如，Murphi 支持用户自定义数据类型、过程以及描述的参数化。

(2) Murphi 描述的结构

Murphi 描述包含常量、类型、全局变量和过程的声明，以及一组转换规则、初始状态描述和一套不变式。

Murphi 的行为部分由一组转换规则构成。每条转换规则都是一个带守卫的命令，包含一个条件（基于全局变量的布尔表达式）和一个动作（可修改变量值的语句）。

条件和动作均采用类 Pascal 语言编写。动作可以是包含循环和条件判断的任意复杂语句，但无论复杂程度如何，动作的执行都是原子性的——执行期间，其他任何规则都无法修改变量或对其造成其他干扰。

(3) Murphi 执行模型

Murphi 状态指为 Murphi 描述中的所有全局变量分配具体值的结果。

Murphi 描述的执行过程通过无限循环以下步骤生成：

无限重复：

- a) 找出当前状态下所有条件为真的规则（即基于全局变量当前值，条件表达式结果为真的规则）。
- b) 任意选择其中一条规则并执行其动作，生成新状态。

验证器生成状态的过程中，会检查各类条件。可能出现多种运行时错误，最典型的包括赋值操作或数组索引时的越界错误。动作中可调用显式的“assert（断言）”和“error（错误）”语句。若出现上述任意一种情况，验证器会停止运行，并输出诊断信息——包含从初始状态到错误状态的完整状态序列重构结果。若当前状态导致

用户在描述中定义的某一不变式表达式为假，或当前状态为“死锁状态”(除自身外无后续状态)，验证器也会执行上述操作。

2. 使用 `./msi -tv` 进行 Debug

在实验过程中，通常需要通过分析不变量违规的执行轨迹来调试协议设计。

由于 Murphi 默认不输出违规轨迹，若需查看错误执行路径，请使用以下命令运行模型：

```
./msi -tv
```

`-tv` 选项用于生成违反任意不变量的最短执行轨迹，便于快速定位问题根源。更多命令行选项可参考 Murphi 用户手册第 2.5 节，其中介绍了多种调试与验证辅助功能。

在分析跟踪记录时，你可能会发现系统的异常并非立即导致不变量违规，而是经过若干步后才被触发，从而生成较长的跟踪序列。

为便于定位问题，建议：

- 可在不修改提供的不变量的前提下，自行添加临时不变量以辅助定位；
- 调试阶段可将 `NUM_CACHE` 从 3 暂时减少为 2，以简化场景并更直观地观察两个处理器间的交互行为。

常见问题

1. Murphi 的解析器不支持 Windows 回车符（CRLF）

在使用 Murphi 编译模型文件时，如果出现如下错误提示：

```
msi.m:4: Bad input character
```

这可能是由于源文件使用了 Windows 格式的行尾符（CRLF）所致。

Murphi 的解析器仅支持 Unix/Linux 格式的行尾符（LF）。

可通过以下命令检查文件的编码格式：

```
file msi.m
```

若输出类似：

```
msi.m: Unicode text, UTF-8 text, with CRLF line terminators
```

则说明当前文件使用的是 **CRLF** 换行符。

请将其转换为 **LF** 格式。

解决方法：

- **方法 1：使用命令行工具**

在 WSL、Git Bash 或 Linux/Mac 终端中执行：

```
dos2unix msi.m
```

若未安装，可通过以下方式安装：

```
sudo apt install dos2unix
```

- **方法 2：在 VSCode 中修改**

1. 打开 `.m` 文件。
2. 右下角状态栏中找到并点击 **"CRLF"** 字样。
3. 在弹出的菜单中选择 **LF**。
4. 保存文件（Ctrl + S）。